DOCUMENT TECHNIQUE

Convention de Codage — V2

Java 21 / Spring Boot 4 — TypeScript / Angular 21+

VVS — Standard technique interne

Version 2.0 — Septembre 2026

Champ	Valeur
Version	2.0 (V2)
Date	Septembre 2026
Portée	Tous projets VVS — Backend Java 21 / Spring Boot 4, Frontend TypeScript / Angular 21+
Architecture	Feature-Based Layering (FBL) — organisation par feature métier, avec un découpage technique interne par couche (controller, domain, dto, mapper, repository, service)
Audience	Développeurs — Débutant à Senior
Classification	Standard technique interne

Ce document constitue le référentiel de référence des normes et bonnes pratiques de développement pour les projets VVS. Il couvre le backend (Java 21, Spring Boot 4, architecture Feature-Based Layering — organisation par feature métier avec un découpage technique interne) et le frontend (TypeScript, Angular 21+), afin de garantir la qualité, la maintenabilité, l'évolutivité et la sécurité des applications.

Changement majeur par rapport à la V1 : la structure du projet n'est pas une architecture Domain-Driven Design (DDD). Elle est corrigée dans ce document en Feature-Based Layering (FBL), alignée sur les conventions réellement en vigueur dans les projets VVS, et les dossiers entity/ sont renommés domain/, avec une séparation service/ (interface) et service/impl/ (implémentation).

1. Conventions de nommage

Le nommage constitue l'un des premiers indicateurs de la qualité d'un code source. Des noms cohérents, explicites et porteurs de sens rendent le code auto-documenté, limitent le recours aux commentaires et facilitent la compréhension, la maintenance et les revues de code.

1.1 Backend — Java / Spring Boot

Élément	Convention	Exemple correct	Exemple incorrect
Classe	PascalCase	OrderService	orderService, order_service
Interface	PascalCase (pas de préfixe I)	OrderRepository	IOrderRepository
Interface + impl	Nom + Impl, dans service/impl/	service/impl/OrderServiceImpl	OrderServiceImplementation
Méthode publique	camelCase	calculateTotal()	CalculateTotal()
Variable locale	camelCase	orderTotal, currentUser	OrderTotal, order_total
Champ privé	camelCase (pas de préfixe _)	orderRepository	_orderRepository, mRepo
Constante (static final)	SCREAMING_SNAKE_CASE	MAX_RETRY_COUNT	MaxRetryCount, maxRetry
Enum (type)	PascalCase singulier	OrderStatus	OrderStatuses
Enum (valeur)	SCREAMING_SNAKE_CASE	IN_PROGRESS	InProgress, inProgress
Record (DTO)	PascalCase + RDto (lecture) / WDto (écriture) ; Dto seul si forme identique en lecture/écriture (référentiel simple)	OrderRDto, CreateOrderWDto, HolidayDto	OrderDTO, OrderRequest
Entité JPA	PascalCase singulier, dans <feature>/domain/	Order, OrderLine	Orders, order_line
Événement de domaine	PascalCase passé	OrderCreatedEvent	OnOrderCreated
Package	tout en minuscule, singulier	com.vvs.order.domain	com.vvs.Orders.Domain
Test	MethodName_Scenario_Expected	createOrder_validData_returnsCreated	testCreateOrder1
Fichier	Même nom que la classe	OrderService.java	orderservice.java
Generic param	T + descriptif	T, ID, TEntity	Type1, T1

1.2 Frontend — TypeScript / Angular

Élément	Convention	Exemple correct	Exemple incorrect
Classe / Composant	PascalCase	OrderListComponent	orderListComponent
Interface / Type	PascalCase	Order, CreateOrderRequest	IOrder, order_dto
Fonction / méthode	camelCase	calculateTotal()	CalculateTotal()
Variable / propriété	camelCase	orderTotal, isLoading	OrderTotal, order_total
Constante globale	SCREAMING_SNAKE_CASE	MAX_PAGE_SIZE	maxPageSize
Champ privé (classe)	camelCase + private	private orderService	private _orderService
Observable	Suffixe $	orders$, isLoading$	orders, ordersObservable
Signal	Nom explicite (pas de suffixe imposé)	orderCount, isLoading	orderCountSignal()
Composant (fichier)	kebab-case, sans suffixe .component	order-list.ts, order-list.html, order-list.css	order-list.component.ts, OrderList.component.ts
Service (fichier)	kebab-case + suffixe -service accolé (pas de point)	order-service.ts	order.service.ts, orderService.ts
Guard / Interceptor / Directive (fichier)	kebab-case + suffixe -guard / -interceptor / -directive	auth-guard.ts, auth-interceptor.ts, has-permission-directive.ts	auth.guard.ts, AuthGuard.ts
Sélecteur composant	kebab-case + préfixe app	app-order-list	orderList, OrderList
Feature dir	kebab-case	order-management/	OrderManagement/
Enum	PascalCase / valeurs PascalCase	OrderStatus.InProgress	OrderStatus.IN_PROGRESS
Test (spec)	describe > it, phrase lisible	"should create an order"	"test1"

1.3 Principes de nommage sémantique

Règle d'or : un nom doit révéler l'intention. Si un nom nécessite un commentaire pour être compris, le nom est mauvais.

Noms obscurs (à éviter)

```
int d; // durée en jours ???
String s; // ???
var list1 = getData(); // quelles données ???
boolean flag; // quel flag ???
void process(); // traiter quoi ???
const temp = calculate(); // temporaire de quoi ???
```

Noms expressifs (attendus)

```
int elapsedDays;
String customerEmail;
List<Order> activeOrders = orderRepository.findActive();
boolean isEligibleForDiscount;
void validatePayment(PaymentRequest payment);
const discountedTotal = calculateDiscountedTotal(order);
```

1.4 Choses à ne jamais faire

- Jamais d'abréviation sauf si universellement connue (Id, Http, Url, Json, Xml, Sql, Dto)
- Jamais de noms à une lettre, sauf dans les lambdas/streams très courts (o -> o.getId())
- Jamais de notation hongroise : strName, intCount, bIsActive
- Jamais de préfixe I sur les interfaces Java (OrderRepository, pas IOrderRepository)
- Jamais de snake_case dans les noms publics Java/TS : get_all_users()
- Jamais de numérotation : handler1, handler2, service3
- Jamais de mot générique seul : Manager, Processor, Data, Info, Helper, Utils
- Jamais de négation dans les booléens : isNotActive → utiliser !isActive ou isInactive

- Jamais any en TypeScript sans justification explicite (// eslint-disable + commentaire)
- Jamais de suffixe .component / .service dans les noms de fichiers Angular : le suffixe de rôle s'accole par un tiret (order-service.ts), jamais par un point (order.service.ts)

2. Organisation et structure du code

2.1 Structure d'une classe Java

Chaque fichier Java suit un ordre strict. Un fichier ne contient qu'une seule classe/interface publique de haut niveau.

```
// === 1. Package ===
package com.vvs.order.service.impl;

// === 2. Imports (groupés, ordonnés, sans wildcard) ===
import com.vvs.order.domain.Order;
import com.vvs.order.repository.OrderRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

// === 3. Classe ===
@Service
public class OrderServiceImpl implements OrderService {

  // 3a. Constantes
  private static final int MAX_ITEMS_PER_ORDER = 100;

  // 3b. Champs privés final (injectés)
  private final OrderRepository orderRepository;
  private final OrderMapper orderMapper;

  // 3c. Constructeur (injection par constructeur uniquement)
  public OrderServiceImpl(OrderRepository orderRepository, OrderMapper orderMapper) {
    this.orderRepository = orderRepository;
    this.orderMapper = orderMapper;
  }

  // 3d. Méthodes publiques
  @Override
  @Transactional
  public OrderRDto create(CreateOrderWDto request) { /* ... */ }

  // 3e. Méthodes privées
  private void validateBusinessRules(Order order) { /* ... */ }
}
```

2.2 Limites de taille

Élément	Limite maximale	Action si dépassé
Méthode	30 lignes	Extraire des méthodes privées
Classe	300 lignes	Découper en classes plus petites (SRP)
Fichier	1 classe/interface publique	Créer un nouveau fichier
Paramètres de méthode	4 paramètres	Créer un objet paramètre (record/DTO)
Profondeur d'imbrication	3 niveaux max	Early return, extraction de méthode
Ligne de code	120 caractères	Retour à la ligne
Composant Angular (template inline)	Extraire au-delà de 15 lignes	Passer en templateUrl séparé
Héritage	2 niveaux max	Préférer la composition

2.3 Organisation Feature-Based Layering (FBL) — Backend

Le code n'est pas organisé par couche technique globale (controller/, service/, repository/ au sommet du projet) mais par feature métier, à plat, en miroir de l'organisation Angular du frontend. Chaque feature regroupe tous les éléments techniques qui la concernent : controller, service (interface + impl), domain, repository, dto, mapper, exception. Ce n'est pas du Domain-Driven Design : il n'y a pas de séparation en couches domain/application/infrastructure, ni de notion formelle d'agrégat ou de bounded context — c'est un découpage par feature avec un layering technique interne simple.

```
com.vvs
├── core                  // Config globale, erreurs globales, utilitaires transverses
│   ├── config            // WebConfig, OpenApiConfig
│   ├── exception         // GlobalExceptionHandler, BaseException
│   └── util              // Utilitaires transverses (dates...)
├── security              // Authentification / autorisation transverses
│   ├── annotation        // @HasPermission
│   ├── config            // SecurityConfig
│   ├── entrypoint        // RestAuthenticationEntryPoint
│   ├── manager           // PermissionManager
│   └── middleware        // JwtFilter
├── shared                // Réutilisable, sans logique métier
│   ├── domain            // BaseEntity (id, createdAt...)
│   ├── dto               // PageResponse, ApiResponse
│   └── mapper            // Utilitaires de mapping communs
├── auth                  // Feature métier
│   ├── controller        // AuthController
│   ├── service           // AuthService (interface)
│   │   └── impl          // AuthServiceImpl
│   ├── domain            // User, UserStatus
│   ├── repository        // UserRepository (Spring Data)
│   ├── dto               // LoginWDto, AuthRDto
│   ├── mapper            // UserMapper (MapStruct)
│   └── exception         // InvalidCredentialsException
└── stock                 // Autre feature, même découpe
    ├── controller
    ├── service
    │   └── impl
    ├── domain
    ├── repository
    ├── dto
    ├── mapper
    └── exception
```

Règle Feature-based
Une feature = un contexte métier cohérent (auth, stock, order, catalog...), autonome et facilement déplaçable/extractible.
core/ et security/ contiennent uniquement le transverse technique (config, gestion d'erreur globale, authentification/autorisation) — jamais de logique métier.
shared/ contient uniquement des briques génériques réutilisées par plusieurs features (entité de base, DTO de pagination) — jamais une entité ou un service propre à une feature.
Une feature ne dépend jamais directement des classes internes d'une autre feature (domain, repository) — elle passe par le service public (interface) exposé, ou par un événement.
À l'intérieur de chaque feature : domain/ porte les entités et les règles métier, service/ (interface) expose le cas d'usage et service/impl/ l'orchestre, controller/ reste une coquille fine, repository/ isole l'accès aux données.

2.4 Organisation Angular — Feature-Based

```
src/app
├── guards                          // auth-guard.ts
├── interceptors                    // auth-interceptor.ts
├── directives                      // has-permission-directive.ts
├── layouts
│   └── main-layout                 // main-layout.ts / .html / .css
├── models                          // Types/interfaces partagés (DTOs frontend)
├── services                        // Services API, un par ressource : order-service.ts
├── shared
│   └── components                  // Composants réutilisables (sidebar...)
├── utils
└── features
    └── order                       // Feature métier (alignée sur la feature backend)
        └── components
            ├── order-list
            │   ├── order-list.ts
            │   ├── order-list.html
            │   ├── order-list.css
            │   └── order-list.spec.ts
            ├── order-detail
            └── order-form
```

- Composants standalone (Angular 21+) — pas de NgModule sauf besoin explicite (librairie tierce).
- Chaque feature regroupe ses composants sous features/<feature>/components/ ; les services d'accès aux données (HttpClient) vivent dans services/, nommés <ressource>-service.ts, alignés 1:1 sur la feature backend correspondante.
- Le state (signals) reste porté par le composant ou un service dédié quand il est partagé entre plusieurs composants : un composant reste un template + une orchestration fine.
- Les routes sont chargées en lazy loading via loadChildren / loadComponent, par feature.

3. Principes SOLID

Les principes SOLID sont les cinq piliers de la conception orientée objet. Ils ne sont pas optionnels ; ils sont la base de tout code maintenable, testable et évolutif, aussi bien côté backend (Java) que côté frontend (services et composants Angular).

3.1 Single Responsibility Principle (SRP)

Principe : une classe (ou un service Angular) ne doit avoir qu'une seule raison de changer.

✕ Violation SRP

```
class OrderService {
  registerOrder() { /* logique métier */ }
  hashPassword() { /* sécurité */ }
  sendConfirmationEmail() { /* SMTP */ }
  generateInvoicePdf() { /* PDF */ }
}
// 4 responsabilités = 4 raisons de changer
```

✓ SRP respecté

```
OrderService         // Orchestration métier des commandes
PasswordHasher       // Hashing des mots de passe
EmailNotifier        // Envoi d'emails
InvoicePdfGenerator  // Génération de PDF
```

3.2 Open/Closed Principle (OCP)

Principe : ouvert à l'extension, fermé à la modification. On étend le comportement via polymorphisme, interfaces ou Strategy, sans toucher au code existant.

```
public interface ShippingCalculator {
  String getMethod();
  BigDecimal calculate(Order order);
}

@Component
public class StandardShipping implements ShippingCalculator { /* ... */ }

@Component
public class ExpressShipping implements ShippingCalculator { /* ... */ }

// Spring injecte automatiquement List<ShippingCalculator>
// Un nouveau mode = une nouvelle classe, zéro modification du code existant
```

3.3 Liskov Substitution Principle (LSP)

Principe : un sous-type doit pouvoir remplacer son type de base sans altérer la correction du programme.

- Violation courante : Repository qui lève UnsupportedOperationException sur certaines méthodes héritées
- Violation courante : Sous-classe qui restreint les préconditions (type de paramètre plus strict)
- Violation courante : Sous-classe qui change les postconditions (retour null au lieu d'exception, ou inversement)

3.4 Interface Segregation Principle (ISP)

Principe : aucun client ne doit dépendre de méthodes qu'il n'utilise pas. Préférer plusieurs interfaces ciblées à une interface monolithique.

```
// ✕ Interface monolithique
public interface DocumentService {
  Document create(...); Document getById(...); List<Document> search(...);
  void validate(...); void reject(...); void archive(...);
  byte[] exportPdf(...); void sendNotification(...);
}

// ✓ Interfaces ciblées
public interface DocumentReader { Document getById(UUID id); List<Document>
search(SearchQuery q); }
public interface DocumentWriter { Document create(CreateDocumentWDto r); }
public interface DocumentWorkflow { void validate(UUID id); void reject(UUID id, String
reason); }
public interface DocumentExporter { byte[] exportPdf(UUID id); }
```

3.5 Dependency Inversion Principle (DIP)

Principe : les modules de haut niveau dépendent d'abstractions, pas d'implémentations concrètes. En Spring, cela se traduit par l'injection par constructeur d'interfaces.

```
// ✓ DIP respecté — injection d'abstractions par constructeur
@Service
public class ReportServiceImpl implements ReportService {
  private final ReportRepository reportRepository; // interface
  private final PdfGenerator pdfGenerator;         // interface
  private final EmailSender emailSender;           // interface

  public ReportServiceImpl(ReportRepository reportRepository,
                           PdfGenerator pdfGenerator,
                           EmailSender emailSender) {
    this.reportRepository = reportRepository;
    this.pdfGenerator = pdfGenerator;
    this.emailSender = emailSender;
  }
  // Testable avec des mocks, interchangeable sans modification
}
```

Règle pratique
Dans un constructeur Spring, chaque paramètre doit être une interface.
Injection par constructeur uniquement — jamais @Autowired sur un champ.
Exceptions acceptées : records/DTOs (données, pas des services), types valeur.

4. Architecture Feature-Based Layering (FBL) — Backend

Chaque feature métier est un package autonome et plat : controller, service (interface dans service/, implémentation dans service/impl/), domain, repository, dto, mapper, exception vivent ensemble dans le même dossier de feature. domain/ porte les entités et les règles métier, le service orchestre le cas d'usage. core/, security/ et shared/ mutualisent le transverse. Il ne s'agit pas de Domain-Driven Design : pas de séparation stricte en sous-packages domain/application/infrastructure/presentation, pas de notion d'agrégat ou de bounded context formalisée.

4.1 Vue d'ensemble des dossiers

Dossier	Rôle	Contient	Règle de dépendance
core/	Transverse technique	Config globale, gestion d'erreur globale, utilitaires	Ne dépend d'aucune feature
security/	Transverse technique	Authentification, autorisation, filtres JWT	Ne dépend d'aucune feature
shared/	Transverse métier générique	BaseEntity (dans shared/domain), DTO génériques (pagination, ApiResponse), mappers communs	Ne dépend d'aucune feature
<feature>/controller	Point d'entrée HTTP	Controllers REST de la feature	Dépend uniquement de <feature>/service
<feature>/service	Cas d'usage	Interface (service/) + implémentation (service/impl/), orchestration métier, transactions	Dépend de <feature>/repository, domain, mapper
<feature>/domain	Modèle métier	Entités JPA porteuses des règles métier	Dépend de shared/ uniquement
<feature>/repository	Accès aux données	Interfaces Spring Data JPA	Dépend de <feature>/domain
<feature>/dto	Contrats d'entrée/sortie	Records (WDto en entrée, RDto en sortie)	Aucune dépendance métier
<feature>/mapper	Conversion	domain ↔ dto (MapStruct)	Dépend de domain + dto
<feature>/exception	Erreurs métier	Exceptions spécifiques à la feature	Étend une exception de base de core/

4.2 Exemple concret — feature auth

```
com.vvs.auth
├── controller
│   └── AuthController.java
├── service
│   ├── AuthService.java        // interface (cas d'usage)
│   └── impl
│       └── AuthServiceImpl.java
├── domain
│   ├── User.java               // porte les règles métier (lock(), etc.)
│   └── UserStatus.java
├── repository
│   └── UserRepository.java     // extends JpaRepository<User, UUID>
├── dto
│   ├── LoginWDto.java          (record)
│   └── AuthRDto.java           (record)
├── mapper
│   └── UserMapper.java         // MapStruct
└── exception
    ├── UserNotFoundException.java
    └── InvalidCredentialsException.java
```

domain — règles métier portées par l'entité

```
// auth/domain/User.java
@Entity
@Table(name = "users")
public class User extends BaseEntity { // BaseEntity vient de shared/domain
  private String email;
  private String passwordHash;

  @Enumerated(EnumType.STRING)
  private UserStatus status;

  public void lock() {
    if (this.status == UserStatus.LOCKED) {
      throw new InvalidUserStateException("Utilisateur déjà bloqué");
    }
    this.status = UserStatus.LOCKED;
  }
}

// auth/repository/UserRepository.java
public interface UserRepository extends JpaRepository<User, UUID> {
  Optional<User> findByEmail(String email);
}
```

service — cas d'usage

```
// auth/service/AuthService.java
public interface AuthService {
  AuthRDto login(LoginWDto request);
}

// auth/service/impl/AuthServiceImpl.java
@Service
public class AuthServiceImpl implements AuthService {
  private final UserRepository userRepository;
  private final PasswordEncoder passwordEncoder;
  private final JwtTokenProvider jwtTokenProvider; // security/

  public AuthServiceImpl(UserRepository userRepository,
                         PasswordEncoder passwordEncoder,
                         JwtTokenProvider jwtTokenProvider) {
    this.userRepository = userRepository;
    this.passwordEncoder = passwordEncoder;
    this.jwtTokenProvider = jwtTokenProvider;
  }

  @Override
  @Transactional
  public AuthRDto login(LoginWDto request) {
    User user = userRepository.findByEmail(request.email())
        .orElseThrow(InvalidCredentialsException::new);
    if (!passwordEncoder.matches(request.password(), user.getPasswordHash())) {
      throw new InvalidCredentialsException();
    }
    return jwtTokenProvider.generate(user);
  }
}
```

controller — coquille fine

```
@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {
  private final AuthService authService;

  public AuthController(AuthService authService) {
    this.authService = authService;
  }

  @PostMapping("/login")
  public ResponseEntity<ApiResponse<AuthRDto>> login(
      @Valid @RequestBody LoginWDto request) {
    AuthRDto result = authService.login(request);
    return ResponseEntity.ok(ApiResponse.success(result, "Connexion réussie"));
  }
}
```

Controller mince
Le controller reçoit la requête HTTP, délègue au service via son interface, retourne la réponse.
Maximum 5 lignes par action. Zéro logique métier, zéro accès direct au repository.
Aucun try/catch — la gestion d'erreur est centralisée dans un GlobalExceptionHandler (core/exception).

4.3 Communication entre features

- Une feature expose son service (interface) comme seul point d'entrée public — les autres features l'injectent, jamais son repository ni son domain directement.
- Pour un couplage faible entre features indépendantes, préférer un événement applicatif (ApplicationEventPublisher) plutôt qu'un appel de service direct.

- Si deux features partagent une notion technique (pagination, réponse API, entité de base), elle migre dans shared/ — jamais dupliquée dans chaque feature.

5. Design Patterns essentiels

5.1 Repository Pattern

- L'interface repository vit dans <feature>/repository/ et est directement implémentée par Spring Data JPA — il n'y a pas de couche infrastructure séparée dans cette organisation.
- Le repository générique (Spring Data JpaRepository) couvre le CRUD standard.
- Les repositories spécialisés ajoutent des méthodes métier (findByStatusAndCreatedAtAfter, etc.).
- Jamais d'injection directe de EntityManager ou du repository Spring Data dans un controller — toujours passer par le service (interface) de la feature.

5.2 Strategy Pattern

Élimine les switch/if-else qui grandissent et respecte l'OCP.

```
public interface PaymentProcessor {
  PaymentMethod getMethod();
  PaymentResult process(Payment payment);
}

@Service
public class PaymentServiceImpl implements PaymentService {
  private final Map<PaymentMethod, PaymentProcessor> processors;

  public PaymentServiceImpl(List<PaymentProcessor> processorList) {
    this.processors = processorList.stream()
        .collect(Collectors.toMap(PaymentProcessor::getMethod, p -> p));
  }

  public PaymentResult process(Payment payment) {
    return Optional.ofNullable(processors.get(payment.method()))
        .orElseThrow(() -> new UnsupportedPaymentMethodException(payment.method()))
        .process(payment);
  }
}
```

5.3 Factory, Specification, Builder

- Factory : encapsule la création d'objets complexes (entités avec invariants à respecter dès la construction).
- Specification : encapsule une règle métier composable, utile pour les filtres de requêtes complexes (JPA Specification API).
- Builder : pour les objets avec de nombreux paramètres optionnels (records privilégiés quand c'est possible).

5.4 Domain Events

Les événements applicatifs découplent les effets de bord (email, audit, notification) de la logique métier centrale.

```
public record UserRegisteredEvent(UUID userId, String email, Instant occurredAt) {}

// Dans le service applicatif
applicationEventPublisher.publishEvent(
    new UserRegisteredEvent(user.getId(), user.getEmail(), Instant.now()));

// Un listener découplé
@Component
public class SendWelcomeEmailListener {
  @EventListener
  @Async
  public void on(UserRegisteredEvent event) { /* envoi email */ }
}
```

5.5 Frontend — Patterns Angular

- Smart / Dumb components : les composants "smart" (containers) orchestrent l'état et les appels service ; les composants "dumb" (présentation) reçoivent des input() et émettent des output() — zéro appel HTTP dans un composant de présentation.
- Facade Pattern : un service façade (order-facade-service.ts) expose une API simple au composant et masque l'orchestration entre plusieurs services.
- Repository Pattern côté front : un service *-service.ts par ressource, seul point d'accès HttpClient pour cette ressource.
- Reactive state avec signals (Angular 21+) : state en lecture seule exposé via signal()/computed(), mutation uniquement via des méthodes dédiées du service.

6. Clean Code — Règles détaillées (Java)

6.1 Gestion des erreurs

Exception	Code HTTP	Usage
DomainException (base)	400 Bad Request	Erreur de validation métier
ResourceNotFoundException	404 Not Found	Ressource introuvable
UnauthorizedException	401 Unauthorized	Non authentifié
ForbiddenException	403 Forbidden	Authentifié mais pas autorisé
ConflictException	409 Conflict	Doublon, contrainte violée
Exception (non gérée)	500 Internal Server Error	Bug — loggé et masqué au client

- Jamais de catch vide : catch (Exception e) {} — interdit
- Jamais de catch qui avale : catch (Exception e) { return null; }
- Toujours propager la cause : throw new DomainException(msg, e), jamais perdre la stack trace
- Les exceptions sont pour les cas exceptionnels, pas le flux normal (préférer Optional<T> pour un résultat absent prévisible)
- Une seule hiérarchie d'exceptions métier par feature, gérée par un @RestControllerAdvice global
- Logger les exceptions avec log.error("message {}", param, exception) — jamais e.printStackTrace()
- Ne jamais exposer les détails techniques au client (stack trace, requête SQL, nom de classe interne)

6.2 Logging structuré

- Utiliser SLF4J (Logger log = LoggerFactory.getLogger(X.class)) — jamais System.out.println
- Templates avec placeholders : log.info("Order {} created for customer {}", ref, customerId) — pas de concaténation
- Ne jamais logger mots de passe, tokens JWT, données personnelles sensibles
- Logger les événements métier importants (création, validation, paiement) en INFO

- Logger les erreurs avec l'exception complète : log.error("message", exception)

6.3 Streams, Optional et Records

- Préférer les Stream API aux boucles impératives pour les transformations de collections, sans les imbriquer à outrance (lisibilité avant tout).
- Optional<T> uniquement en valeur de retour, jamais en paramètre de méthode ni en champ d'entité.
- orElseThrow() avec une exception métier explicite plutôt que orElse(null).
- Utiliser les records Java pour tous les DTOs (immutables par nature, thread-safe).
- var autorisé quand le type est évident à la lecture, sinon type explicite.

```
// ✓ Record pour un DTO immuable
public record CreateOrderWDto(
    @NotBlank String customerId,
    @NotEmpty @Valid List<OrderLineWDto> lines
) {}

public record OrderRDto(UUID id, String reference, BigDecimal total) {}
```

6.4 Null safety

- Annoter les paramètres/retours avec @NonNull / @Nullable (JSpecify ou org.springframework.lang) quand la nullité n'est pas évidente.
- Ne jamais retourner null pour une collection — retourner une collection vide.
- Préférer Optional.ofNullable(...).orElseThrow(...) au if (x == null) imbriqué.
- Valider les entrées au plus tôt avec Bean Validation (@NotNull, @NotBlank, @Valid).

7. API RESTful — Conventions

7.1 Nommage des endpoints

Les URL utilisent des noms au pluriel, en kebab-case. Les verbes HTTP définissent l'action ; les URL ne contiennent jamais de verbe.

Verbe	Route	Description	Code succès
GET	/api/v1/orders	Liste paginée avec filtres	200 OK
GET	/api/v1/orders/{id}	Détail d'une ressource	200 / 404
POST	/api/v1/orders	Créer une ressource	201 Created
PUT	/api/v1/orders/{id}	Remplacer une ressource complète	200 / 404
PATCH	/api/v1/orders/{id}	Modifier partiellement	200 / 404
DELETE	/api/v1/orders/{id}	Supprimer une ressource	204 No Content
GET	/api/v1/orders/{id}/lines	Sous-ressources	200
POST	/api/v1/orders/{id}/validate	Action sur une ressource	200

7.2 Codes HTTP

Code	Nom	Usage
200	OK	GET, PUT, PATCH réussis
201	Created	POST réussi (+ header Location)
204	No Content	DELETE réussi
400	Bad Request	Validation échouée, données invalides
401	Unauthorized	Non authentifié (token absent/expiré)
403	Forbidden	Authentifié mais permission insuffisante
404	Not Found	Ressource inexistante
409	Conflict	Doublon, violation de contrainte
422	Unprocessable Entity	Données valides mais logiquement incorrectes
429	Too Many Requests	Rate limit dépassé
500	Internal Server Error	Bug serveur (loggé, détails masqués)

7.3 Réponse standardisée

```
public record ApiResponse<T>(
    boolean success,
    String message,
    T data,
    Instant timestamp
) {
  public static <T> ApiResponse<T> success(T data, String msg) {
    return new ApiResponse<>(true, msg, data, Instant.now());
  }
  public static <T> ApiResponse<T> fail(String msg) {
    return new ApiResponse<>(false, msg, null, Instant.now());
  }
}
```

7.4 Pagination obligatoire

Tout endpoint GET retournant une liste DOIT être paginé (Spring Data Pageable). Un endpoint sans pagination qui retourne des dizaines de milliers de lignes est un incident en production.

```
@GetMapping
public ResponseEntity<ApiResponse<Page<OrderRDto>>> getAll(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(required = false) String search,
    @RequestParam(defaultValue = "createdAt,desc") String sort) {
  Pageable pageable = PageRequest.of(page, size, Sort.by(parseSort(sort)));
  return ResponseEntity.ok(
      ApiResponse.success(orderService.getAll(search, pageable), "OK"));
}
```

7.5 Versioning et attributs obligatoires

- Versionner via le chemin URL : /api/v1/..., /api/v2/... pour les breaking changes.
- @RestController + @RequestMapping("/api/v1/...") systématique.
- @Valid sur tout @RequestBody entrant.
- @PreAuthorize ou configuration Spring Security par défaut sur tous les endpoints (deny by default).

8. Spring Data JPA — Bonnes pratiques

8.1 Configuration des entités

- Une entité JPA par fichier, dans <feature>/domain/.

- Spécifier explicitement @Column(nullable, length), @Index, et le FetchType (LAZY par défaut sur les relations).
- Configurer explicitement le CascadeType et l'orphanRemoval — jamais de comportement implicite non documenté.
- Utiliser @Version pour le verrouillage optimiste sur les entités sensibles à la concurrence.

8.2 Requêtes — Règles obligatoires

- @Transactional(readOnly = true) pour toute méthode de lecture seule.
- Utiliser des projections (interfaces ou DTO records) pour ne charger que les colonnes nécessaires.
- JOIN FETCH explicite pour éviter le N+1 — jamais de lazy loading implicite déclenché en dehors de la transaction.
- Pageable pour paginer — jamais findAll() sans limite sur une table volumineuse.
- Requêtes @Query nommées et documentées plutôt que des noms de méthode dérivés à rallonge.

✕ Anti-patterns JPA

```
List<Order> orders = orderRepository.findAll(); // charge tout en mémoire
orders.forEach(o -> o.getLines().size());       // N+1 query

@Transactional
public List<Order> findActive() { ... } // lecture marquée en écriture
```

✓ Pattern correct

```
@Transactional(readOnly = true)
public Page<OrderRDto> getActive(Pageable pageable) {
  return orderRepository.findByStatus(OrderStatus.ACTIVE, pageable)
      .map(orderMapper::toRDto);
}
```

8.3 Migrations (Flyway/Liquibase)

- Les scripts de migration sont versionnés dans le repository Git, jamais modifiés une fois appliqués en production.
- Nommer les migrations de manière descriptive : V12__add_locked_until_to_users.sql.
- Une migration = un changement logique cohérent, testé en local avant merge.

9. Sécurité

9.1 Authentification

- Hashing exclusivement avec BCrypt via Spring Security PasswordEncoder (jamais MD5, SHA256, texte clair).
- Access token JWT à durée courte (15-30 min) + refresh token long (7 jours).
- Refresh token stocké en base et révocable à la déconnexion.
- Rotation du refresh token à chaque utilisation, détection de réutilisation → révocation de tous les tokens de l'utilisateur.
- Blocage de compte après N tentatives échouées (configurable, typiquement 5).
- Aucune tolérance d'horloge (clock skew) non justifiée dans la validation JWT.

9.2 Autorisation

- Deny by default : toute route protégée sauf déclaration explicite (permitAll sur login, register, forgot-password uniquement).

- Permissions incluses dans le JWT sous forme de claims / authorities Spring Security.
- @PreAuthorize("hasAuthority('ORDER_WRITE')") pour le contrôle fin au niveau méthode.
- Ne jamais exposer d'identifiants séquentiels dans les URL — utiliser des UUID.

9.3 Protection des données

- Jamais de mot de passe, token ou secret dans les logs.
- Jamais de secret dans le code source — variables d'environnement ou vault (jamais commit dans application.yml).
- CORS avec origines explicites — pas de wildcard * en production.
- Validation côté serveur obligatoire, même si le frontend valide déjà.
- Requêtes paramétrées uniquement (JPQL/Criteria) — jamais de concaténation de chaînes SQL.
- Headers de sécurité HTTP : X-Content-Type-Options, X-Frame-Options, CSP, HSTS.
- Rate limiting sur les endpoints sensibles (login, reset password).
- Message d'erreur identique au login que l'email existe ou non (pas d'énumération de comptes).

9.4 Sécurité côté Angular

- Aucun secret (clé API, credentials) dans le code frontend — tout secret reste côté backend.
- Sanitizer systématique des entrées affichées en HTML (DomSanitizer avec parcimonie, éviter [innerHTML] non maîtrisé).
- Stockage du token en mémoire ou cookie HttpOnly plutôt que localStorage quand le contexte de sécurité l'exige.
- Un HttpInterceptor centralisé (auth-interceptor.ts) pour l'ajout du token et la gestion du refresh, jamais dupliqué par service.
- Guards de routes (CanActivateFn, ex. auth-guard.ts) pour la protection des routes selon rôle/permission.

10. Frontend — Bonnes pratiques Angular / TypeScript

10.1 Composants

- Composants standalone systématiques (Angular 21+) — imports explicites, pas de NgModule inutile.
- changeDetection: ChangeDetectionStrategy.OnPush par défaut sur tous les composants.
- Un composant = une responsabilité d'affichage ; la logique métier reste dans un service.
- Utiliser les nouveaux control flow (@if, @for, @switch) plutôt que *ngIf/*ngFor.
- input() / output() (API signals) plutôt que @Input()/@Output() décorateurs sur les nouveaux composants.

10.2 Services et injection

- providedIn: 'root' par défaut pour les services applicatifs transverses.
- inject() pour l'injection dans les fonctions/guards/resolvers, constructeur pour les classes.
- Un service = une responsabilité (API client, mapping, état) — pas de service fourre-tout.

```
@Injectable({ providedIn: 'root' })
export class OrderService {
  private readonly http = inject(HttpClient);
  private readonly baseUrl = '/api/v1/orders';

  getAll(params: OrderQuery): Observable<Page<OrderRDto>> {
    return this.http.get<Page<OrderRDto>>(this.baseUrl, { params: toHttpParams(params)
});
  }

  create(request: CreateOrderWDto): Observable<OrderRDto> {
    return this.http.post<OrderRDto>(this.baseUrl, request);
  }
}
```

10.3 RxJS et Signals

- Toujours désabonner (async pipe, takeUntilDestroyed()) — jamais de subscribe() sans gestion du cycle de vie.
- Préférer les opérateurs de transformation (map, switchMap, filter) à la logique manuelle dans subscribe().
- switchMap pour les requêtes qui annulent la précédente (recherche), mergeMap/concatMap pour les cas parallèles/séquentiels.
- Signals pour l'état local synchrone simple ; RxJS pour les flux asynchrones complexes (WebSocket, debounce, combineLatest).
- Pas de mutation directe d'un signal partagé depuis un composant — passer par une méthode du service.

✕ Anti-pattern RxJS
this.orderService.getAll().subscribe(orders => this.orders = orders);
// pas de désabonnement -> fuite mémoire potentielle

✓ Pattern correct
orders = toSignal(this.orderService.getAll(), { initialValue: [] });
// ou avec takeUntilDestroyed() dans un constructeur/effect

10.4 Typage strict

- strict: true dans tsconfig.json — aucune régression tolérée.
- Interdiction de any sauf cas exceptionnel documenté ; préférer unknown + narrowing.
- Un type/interface par DTO backend, généré ou maintenu en miroir strict du contrat API.
- Types explicites sur les retours de fonction publique, inférence acceptée en interne.

11. Tests

11.1 Pyramide des tests

Type	Quantité	Cible	Outils
Tests unitaires	Majorité (70%)	Services, logique métier, mappers	JUnit 5 + Mockito + AssertJ / Jasmine + Karma ou Jest
Tests d'intégration	Moyen (20%)	Repositories, controllers, pipeline complet	@SpringBootTest, Testcontainers / TestBed Angular
Tests E2E	Minimum (10%)	Scénarios utilisateur complets	Playwright, Cypress

11.2 Convention de nommage — Backend (JUnit 5)

Format : methodName_scenario_expectedBehavior

```
createOrder_validData_returnsCreatedOrder()
createOrder_emptyLines_throwsDomainException()
login_validCredentials_returnsTokens()
login_wrongPassword_throwsUnauthorizedException()
login_lockedAccount_throwsForbiddenException()
```

11.3 Structure AAA (Java)

```
@Test
void createOrder_validData_returnsCreatedOrder() {
  // Arrange
  var request = new CreateOrderWDto("CUST-001", List.of(new OrderLineWDto("PROD-1", 2)));
  when(orderRepository.save(any())).thenAnswer(inv -> inv.getArgument(0));

  // Act
  OrderRDto result = orderService.create(request);

  // Assert
  assertThat(result).isNotNull();
  assertThat(result.reference()).isNotBlank();
  verify(orderRepository).save(any(Order.class));
}
```

11.4 Tests Angular (Jasmine/Jest + TestBed)

```
describe('OrderListComponent', () => {
  it('should display orders returned by the service', () => {
    // Arrange
    const orderServiceSpy = jasmine.createSpyObj('OrderService', ['getAll']);
    orderServiceSpy.getAll.and.returnValue(of({ content: [mockOrder], totalElements: 1
}));

    // Act
    const fixture = TestBed.configureTestingModule({
      imports: [OrderListComponent],
      providers: [{ provide: OrderService, useValue: orderServiceSpy }],
    }).createComponent(OrderListComponent);
    fixture.detectChanges();

    // Assert
    expect(fixture.nativeElement.querySelectorAll('.order-row').length).toBe(1);
  });
});
```

11.5 Règles de tests

- Un test = un scénario = un assert logique.
- Les tests sont indépendants et exécutables dans n'importe quel ordre.
- Pas de dépendance externe (DB réelle, réseau) en tests unitaires — mocks/stubs uniquement.
- Couvrir les cas positifs, négatifs et les cas limites (edge cases).
- Tests rapides (< 100ms en unitaire).
- @ParameterizedTest + @CsvSource / @MethodSource pour les tests paramétrés côté Java.
- Viser 80% de couverture minimum sur les services métier et les composants critiques du parcours utilisateur.

12. Grille de revue de code

12.1 Sévérité P0 — Bloquant (merge impossible)

- Violation de l'architecture Feature-Based (logique métier dans core/security/shared, feature qui accède au domain/repository interne d'une autre feature)
- Faille de sécurité (injection SQL, XSS, énumération d'emails, mot de passe en clair)
- TODO / throw new UnsupportedOperationException() laissé dans une PR
- EntityManager ou Repository Spring Data injecté directement dans un controller (violation DIP)
- System.out.println / console.log au lieu du logger structuré
- Code qui ne compile pas / build qui échoue
- Duplication de code > 20 lignes
- Valeurs sensibles hardcodées au lieu d'une configuration/secret manager
- Endpoint GET sans pagination retournant une liste potentiellement volumineuse
- catch vide qui avale les exceptions (Java) / subscribe() sans gestion d'erreur (Angular)

12.2 Sévérité P1 — Majeur (à corriger sous 24h)

- Absence de @Transactional(readOnly = true) sur une méthode de lecture seule
- Pas de validation Bean Validation (@Valid) sur un DTO d'entrée
- Pas de logger dans un service qui devrait tracer des événements métier
- Valeurs en dur au lieu de @ConfigurationProperties / environment.ts
- Endpoint sans protection Spring Security explicite
- N+1 query détectée (JPA) ou appel HTTP en boucle (Angular)
- Pas de test unitaire sur un service métier critique
- Exception générique (throw new RuntimeException("...")) au lieu d'une exception métier typée
- Méthode/fonction > 30 lignes sans justification
- Composant Angular sans ChangeDetectionStrategy.OnPush sans raison

12.3 Sévérité P2 — Mineur (suggestion)

- Commentaires inutiles ou sur-verbeux
- Code mort ou commenté
- Nommage non conforme aux conventions du présent document
- Import inutilisé
- Classe > 300 lignes
- Manque de Javadoc/TSDoc sur les méthodes publiques critiques

12.4 Checklist complète avant PR

Catégorie	Vérification
Compilation	Build backend et frontend sans warning, 0 erreur
Architecture	Code dans le bon dossier de feature (FBL), core/security/shared sans logique métier
SOLID - SRP	Chaque classe/service a une seule responsabilité
SOLID - DIP	Constructeurs n'exposent que des interfaces
Validation	Chaque DTO d'entrée est validé (@Valid / formulaire réactif Angular)
Sécurité	Endpoints protégés par défaut, permissions vérifiées
Logging	Logger structuré pour les événements importants, rien de sensible loggé
Tests	Tests unitaires écrits (cas positifs + négatifs)
Configuration	application.yml/environment.ts sans secret en dur
DRY	Pas de code dupliqué > 10 lignes
Pagination	Toutes les listes exposées par l'API sont paginées
JPA	readOnly sur les lectures, JOIN FETCH explicite, pas de N+1
API	Codes HTTP corrects, ApiResponse<T> standardisé
Clean Code	Méthodes < 30 lignes, classes < 300 lignes
Nommage	Conventions respectées (PascalCase, camelCase, kebab-case selon le contexte, RDto/WDto)
Angular	Composants standalone, OnPush, désabonnement RxJS géré, fichiers sans suffixe à point